

Gmail Cleanup Strategy — Technical Spec

2026-09-20 · @Someone

Overview

Mark's Gmail account holds 412,847 messages (61.2 GB) collected since August 2005, now billed under Google's storage pricing. The goal is a cleanup tool that ranks senders from safest to riskiest to delete and lets Mark approve every batch himself — nothing is ever auto-deleted.

Deletion is also staged, not immediate: an approved batch is labeled and archived first, with a 30-day restore window before anything moves to permanent deletion.

Data schema

Metadata only — never message bodies — pulled once and stored locally in SQLite.

messages

Column	Type	Notes
message_id	text (PK)	Gmail message id
thread_id	text	groups replies in a thread
sender_email	text	full From address
sender_domain	text	derived, used for grouping
subject	text	
date	integer	epoch, from the Date header
size_bytes	integer	
is_read	boolean	
is_starred	boolean	
has_attachment	boolean	
labels	text	JSON array of Gmail label ids

Column	Type	Notes
category	text	assigned by the scoring pass

sender_groups (derived by aggregating `messages` on `sender_domain`)

Column	Type	Notes
sender_domain	text (PK)	
category	text	
message_count	integer	
total_size_bytes	integer	
first_seen	integer	
last_seen	integer	
pct_unread	real	
pct_attachments	real	
pct_starred	real	
delete_safety_score	real	computed, see Scoring model
approval_status	text	pending / approved / rejected / skipped

Scoring model

Every sender group gets a 0-100 `delete_safety_score`; higher means safer to delete. Scoring runs at the group level, not per message.

Category weights (base points out of 40)

Category	Weight
Spam / phishing leftovers	40
Marketing / promotional	35
Automated notifications	35

Category	Weight
Newsletters / subscriptions	25
Transactional / receipts	15
Business-critical	0
Personal correspondence	0

```

def delete_safety_score(group):
    score = CATEGORY_WEIGHT[group.category]                # 0-40
    score += sender_pattern_weight(group)                   # 0-20
    score += age_weight(group)                              # 0-20
    score -= attachment_penalty(group)                      # 0-20
    score += size_bonus(group)                             # 0-10
    if group.pct_starred > 0 or group.has_protected_label:
        return 0                                           # hard
override: never auto-safe
    return max(0, min(100, score))

def sender_pattern_weight(group):
    # more weight for high, steady volume and a sender that has gone quiet
    volume = min(group.message_count / 5000, 1) * 12
    dormant = 8 if (TODAY - group.last_seen).days > 365 else 0
    return volume + dormant

def age_weight(group):
    avg_age_years = (TODAY - group.avg_date).days / 365
    return min(avg_age_years / 6, 1) * 20

def attachment_penalty(group):
    return group.pct_attachments * 20                      #
attachments pull the score down hard

def size_bonus(group):
    # bigger space savings ranks higher for review priority, not deletion
    itself
    return min(group.total_size_bytes / (2 * 1024**3), 1) * 10

```

Read/unread status is not scored directly. It is shown alongside the sample messages in the review screen, since its meaning flips by category — unread marketing suggests no engagement, but unread personal mail may mean something was missed.

Extraction architecture

A Python script pulls metadata only, in batches, and writes it into the local `messages` table so the whole 412k-message pull can pause and resume without re-fetching what's already stored.

```

flowchart LR
    A[OAuth: gmail.readonly<br/>metadata scope] --> B[users.messages.list<br/>paginate by pageToken]
    B --> C[users.messages.batchGet<br/>format=metadata]
    C --> D[(SQLite: messages)]
    D --> E[Checkpoint:<br/>last pageToken]
    E -->|resume on restart| B
  
```

- **Auth & scope:** use `gmail.metadata` (read-only, headers and labels only) for the extraction pass; a separate, later step re-authenticates with `gmail.modify` only when Mark approves the first batch of label/archive actions.
- **Pagination:** `users.messages.list` returns up to 500 ids per page; batch-fetch each page's metadata with `format=metadata` and `metadataHeaders=[From, Subject, Date]` to avoid downloading bodies.
- **Rate limits:** Gmail's API allows roughly 250 quota units/user/second; batch requests (up to 100 per HTTP call) keep this well under budget even at 412k messages.
- **Checkpointing:** store the last successful `pageToken` and message count in a `run_state` table; a crashed or interrupted run resumes from there instead of restarting.
- **Sender grouping:** after the raw pull, one aggregation pass builds `sender_groups` from `messages`, computing the fields the scoring model needs.

Review workflow

Approvals happen at the sender-group level; execution always follows the same three-step safety pattern, never a direct delete.

```

stateDiagram-v2
    [*] --> Pending
    Pending --> Approved: Mark approves group
    Pending --> Rejected: Mark rejects
    Pending --> Skipped: decide later
    Approved --> Labeled: batchModify<br/>add Cleanup/Archive,<br/>remove INBOX
    Labeled --> RestoreWindow: 30-day timer starts
    RestoreWindow --> Restored: Mark restores
    RestoreWindow --> PermanentDelete: window closes +<br/>Mark confirms again
    Restored --> [*]
    PermanentDelete --> [*]

```

- **Label, then archive, then delete:** an approved group is first modified with `users.messages.batchModify` — add a `Cleanup/Archive` label, remove `INBOX` — in chunks of up to 1,000 ids per call. Nothing is deleted at this step.
- **Restore window:** for 30 days the labeled messages stay in the account, findable by label, and a one-click restore reverses the label change.
- **Permanent delete:** only after the window closes, and only with a second explicit confirmation per group, does `users.messages.batchDelete` run.
- **Batch approval UI:** the review screen groups by sender, shows the score breakdown and a sample of real subject lines, and lets Mark approve, reject, or skip each group before it ever reaches the approval queue.

UI/UX prototype

A four-screen clickable prototype covers the whole flow: [Gmail Cleanup — Review App Prototype](#).

Screen	Shows	Key interaction
Dashboard	Sender groups ranked by score, with category filter chips, message count, size, and active date range	Select groups, jump into a group's review
Group review	One sender's score breakdown (each factor as its own bar) plus a sample of real subject lines with read/unread and attachment status	Reject, skip, or approve the group

Screen	Shows	Key interaction
Approval queue	The batch about to be actioned — every approved group, total count and size, plain-language explanation of what happens next	Confirm and start the archive run
Progress	Live batch status, storage reclaimed so far, and a per-group restore countdown	Restore any archived group before its window closes

The design deliberately keeps message-level review rare: Mark acts on sender groups, and only drops into individual messages when a group’s sample looks ambiguous.

Handoff notes

Build in this order: extraction, then scoring, then the review UI, then execution. Each stage is runnable and checkable on its own before the next depends on it.

```
gmail-cleanup/  
  extract/          # Gmail API pull -> SQLite (Data schema, Extraction  
architecture)  
  score/           # delete_safety_score and its inputs (Scoring model)  
  review-ui/       # local app implementing the four screens in the prototype  
link above  
  execute/         # batchModify / batchDelete runner, gated by the approvals  
table (Review workflow)  
  data/            # local SQLite file, gitignored
```

Suggested split: Claude Code is well-placed for `extract/`, `score/`, and `execute/` — it can authenticate to the Gmail API directly and iterate against Mark’s real mailbox, which this chat cannot reach. Codex can take the prototype link above as a visual reference and build out `review-ui/`.

Guardrails to hard-code, not leave to runtime judgment:

- Label-and-archive before any delete call is ever reachable in code — no path skips straight to `batchDelete`.
- Every batch action requires a stored approval row before it runs; the executor refuses to act on an unapproved group.
- Dry-run mode on by default; a real run needs an explicit flag.

- Cap batch size per run (for example 5,000 messages) so a bug can't silently process the whole account in one pass.
- Permanent deletion requires a second, separate confirmation after the 30-day window, never bundled with the original approval.